Contents

High-Level Design Document **1**
Contingency Deadline Copilot — AI Real Estate Transaction Document Intelligence Assistant . . . 1
1. Purpose . . . 1
2. Goals and Non-Goals . . . 1
3. System Overview . . . 2
4. Component Breakdown . . . 4
5. Data Flow — Extraction Sequence . . . 5
6. Product State Machine . . . 6
7. Data Model (Planned) . . . 6
8. Technology Stack . . . 6
9. Key Design Decisions . . . 7
10. Open Risks and Mitigations . . . 7
11. Current Implementation Status (as of this document) . . . 8
12. Next Steps . . . 8

High-Level Design Document

Contingency Deadline Copilot — AI Real Estate Transaction Document Intelligence Assistant

Version: 1.0 **Date:** September 2026 **Status:** In Development (v1 / MVP)

---

1. Purpose

This document describes the high-level architecture, system components, data flow, and technical design decisions for the Contingency Deadline Copilot — a tool that extracts contingency deadlines from real estate purchase agreements and computes accurate calendar dates from relative contract terms (e.g. "10 business days after acceptance"), with human-verified alerts.

This HLD covers the v1 (narrowed) scope as defined in the product PRD: a single document type (purchase agreements), a single job (deadline extraction and tracking), for solo agents and small transaction-coordination teams.

---

2. Goals and Non-Goals

Goals (v1)

- Extract contingency clauses and their deadlines from purchase agreements with source citations
- Convert relative date language into exact, holiday-aware calendar dates
- Require human confirmation before any alert is scheduled
- Maintain a defensible audit trail of extractions, confirmations, and edits
- Support per-contract pricing with no login required to see first value

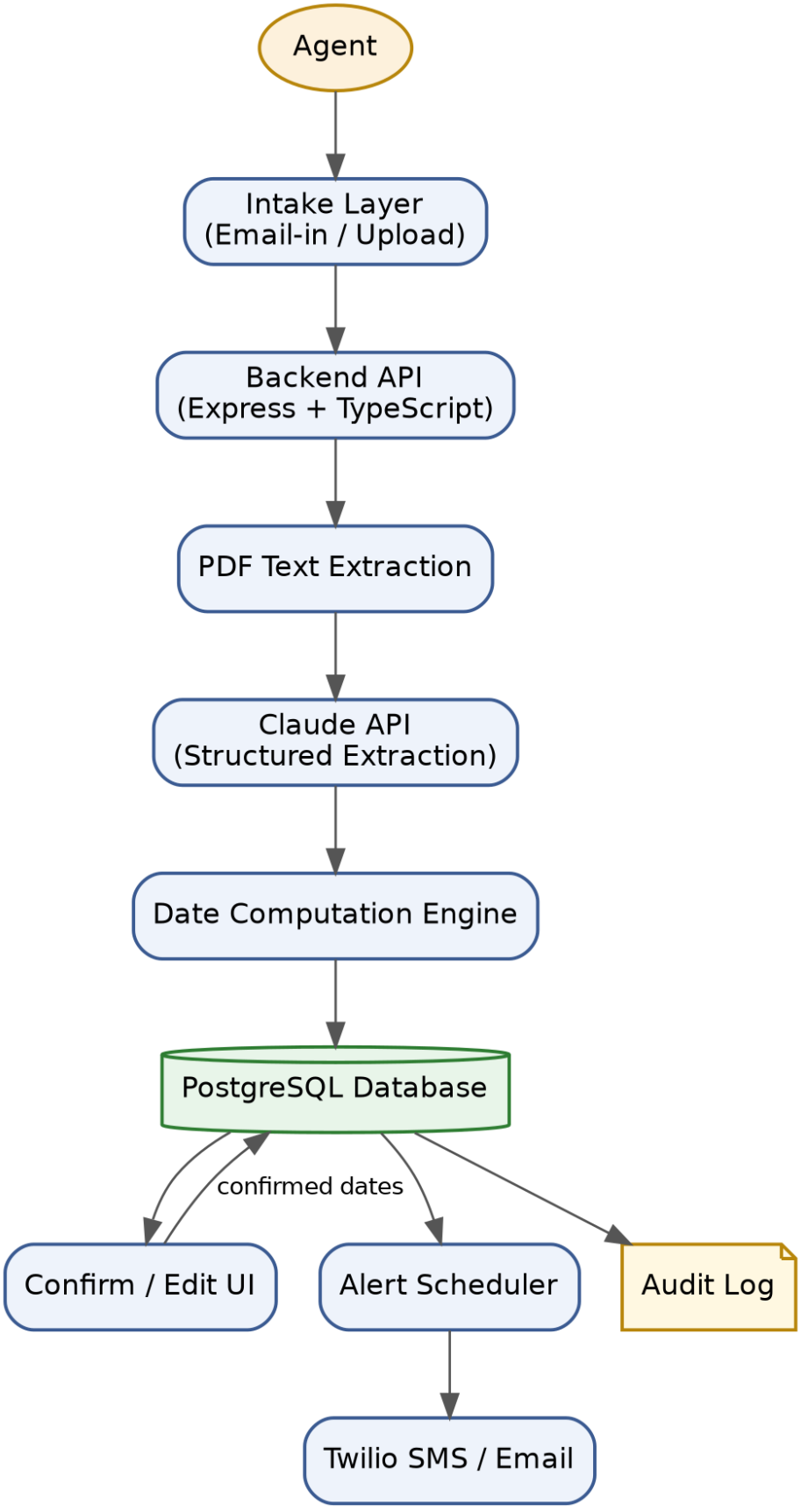
Non-Goals (v1 — deferred to v2+)

- Additional document types (disclosures, inspection reports, loan documents)
- Portfolio-wide, multi-deal dashboards
- ML-based risk prediction or compliance analysis
- E-signature platform integrations
- Multi-user team accounts / role-based permissions
- TC platform integrations (Dotloop, SkySlope, Qualia)

---

3. System Overview

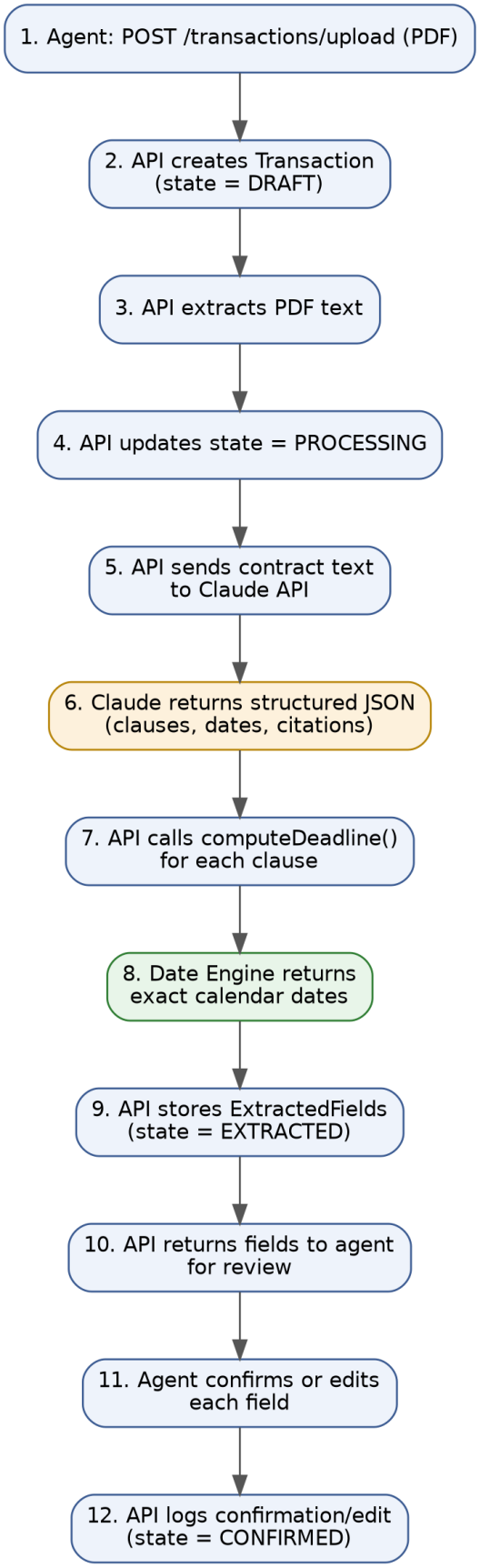
The system is a web application with three logical layers: a client-facing frontend for upload/review/confirmation, a backend API that orchestrates extraction and date computation, and supporting services for AI extraction, storage, and notifications.


Agent
Intake Layer
(Email-in / Upload)
Backend API
(Express + TypeScript)
PDF Text Extraction
Claude API
(Structured Extraction)
Date Computation Engine
PostgreSQL Database
confirmed dates
Confirm / Edit UI
Alert Scheduler
Audit Log
Twilio SMS / Email

4. Component Breakdown

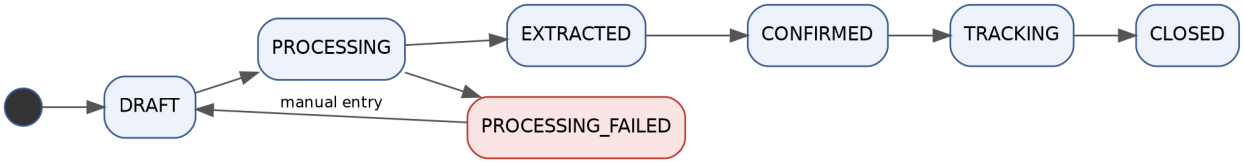
Component	Responsibility	Status
Intake Layer	Email-in (Mailgun/SendGrid) + direct file upload	Not started
Backend API	Express + TypeScript; orchestrates the extraction/confirmation flow	In progress — `/health` and `/extract` endpoints built
PDF Text Extraction	Converts uploaded PDF into raw text for AI processing	Not started
AI Extraction Service	Calls Claude API to extract clauses, dates, and source citations as structured JSON	Built and tested in isolation; wired into `/extract` endpoint with mock fallback
Date Computation Engine	Converts relative date terms into exact calendar dates, business/calendar-day and holiday aware (US + India)	**Complete** — built, unit tested, timezone bug found and fixed
Database	Stores transactions, extracted fields, confirmations, and audit log	Not started — Prisma + PostgreSQL planned
Confirm/Edit UI	Human-in-the-loop review of extracted dates against source PDF	Not started
Alert Scheduler	Triggers SMS/email reminders ahead of confirmed deadlines	Not started
Audit Log	Immutable record of every extraction, confirmation, and edit	Not started (schema design pending)

5. Data Flow — Extraction Sequence



6. Product State Machine

Per the PRD, a transaction moves through a linear lifecycle:



`PROCESSING_FAILED` is the designated failure state for unreadable or non-standard documents, routing to manual entry rather than silently failing — per the "fail loud, not silent" product principle.

7. Data Model (Planned)

Core entities, based on the product's v1 scope:

Transaction - id, status (enum matching state machine), source_file_url, acceptance_date, created_at, updated_at

ExtractedField - id, transaction_id (FK), clause_type, number_of_days, day_type, source_text, source_page, confidence, computed_deadline, confirmed (boolean), confirmed_at

AuditLogEntry - id, transaction_id (FK), field_id (FK), action (extracted/confirmed/edited), old_value, new_value, timestamp

This schema directly supports US-004 (confirm/correct) and US-007 (audit trail) from the PRD.

8. Technology Stack

Layer	Technology	Rationale
Backend	Node.js, Express, TypeScript	Type safety for structured extraction data; strong Prisma integration
Database	PostgreSQL + Prisma ORM	Relational fit for transaction/audit data; type-safe queries
AI Extraction	Claude API (`@anthropic-ai/sdk`)	Structured JSON output with source grounding
Date Logic	`date-fns`, `date-holidays`	Business-day math, US + India holiday calendars
PDF Processing	`pdf-parse` (planned), OCR fallback (Tesseract/Textract)	Text-layer extraction with a path for scanned documents
File Storage	Local disk (dev) → S3/R2 (prod)	Simple dev loop, scalable path to production

Layer	Technology	Rationale
Alerts	Twilio (SMS), SES/SendGrid (email), node-cron	Standard, well-documented integrations
Payments	Stripe Checkout	Per-contract pricing matches PRD's deal-based pricing model
Testing	Jest + ts-jest	Type-checked test execution
Frontend (planned)	Next.js, Tailwind, pdf.js	Source-highlighted PDF review UI

---

9. Key Design Decisions

Date engine built and validated first, independent of AI/database. Per the PRD's "narrow before broad" principle, the date computation logic — the most defensible and highest-risk-if-wrong piece of the product — was built and unit-tested in isolation before any infrastructure was added around it.

Mock-mode fallback for AI extraction. The extraction service automatically falls back to a realistic mocked response when no API key is present, and switches to live Claude calls once a key is configured — no code changes required. This decouples early development/demo work from API cost/availability.

Timezone-safe holiday comparison. An early implementation compared dates using the `date-holidays` library's UTC-based range checks, which produced incorrect results depending on the host machine's local timezone. This was fixed by comparing holiday dates as plain `YYYY-MM-DD` strings instead — a deliberate design choice to avoid timezone ambiguity in a domain (legal deadlines) where correctness is non-negotiable.

Human-in-the-loop by design, not as an afterthought. No alert is ever scheduled from an unconfirmed date (Product Principle #3). This is enforced at the data model level — the `confirmed` flag gates alert scheduling — not just at the UI level.

---

10. Open Risks and Mitigations

Risk	Mitigation
AI extraction accuracy below 95% target	Confidence flagging on every extracted field; low-confidence fields routed to manual review
Scanned/non-text PDFs fail extraction	OCR fallback planned; `PROCESSING_FAILED` state routes to manual entry rather than silent failure
Holiday calendar library discrepancies across regions	Validated via targeted unit tests; timezone bug already caught and fixed pre-launch
API cost/availability during early development	Mock-mode fallback decouples core pipeline development from live API dependency

---

11. Current Implementation Status (as of this document)

- [DONE] Date computation engine — built, unit tested (4/4 passing), US + India holiday support
- [DONE] TypeScript + Express backend foundation — `/health` endpoint verified working
- [DONE] AI extraction service — built with mock fallback, wired to `/extract` endpoint
- [IN PROGRESS] Database schema — designed above, not yet implemented (Prisma + PostgreSQL)
- [NOT STARTED] PDF upload and text extraction — not started
- [NOT STARTED] Confirm/edit frontend — not started
- [NOT STARTED] Alert scheduling — not started
- [NOT STARTED] Payments — not started (deliberately last, per build sequencing)

---

12. Next Steps

1. Implement Prisma schema and run first migration against hosted PostgreSQL (Supabase)
2. Build `POST /transactions/upload` and wire PDF text extraction
3. Connect the full pipeline: upload → extract → store → confirm → retrieve
4. Build minimal confirm/edit frontend for live demonstration